



DETOX PASS

PLAYBOOK DE DESENVOLVIMENTO

Git, revisao de codigo, ambientes, Supabase, CI/CD, integracoes e regras para Cursor/IA

Objetivo deste documento

Este playbook e o acordo operacional do time formado por Samuel, Elias e Anderson. Ele tambem deve ser anexado ao Cursor para que os agentes de IA sigam as mesmas regras dos humanos. O objetivo e desenvolver rapido, com ajuda de IA, sem transformar velocidade em bagunca, perda de codigo ou risco para producao.

Pessoa	Foco principal	Quando precisa revisar
Samuel	Governanca do kickoff, repositorio/documentacao, arquitetura, produto, contas/ambientes, pagamentos, seguranca e releases	Define a base do projeto com Elias e revisa Stripe, auth, RLS, migrations, producao e decisoes estruturais
Elias	Bootstrap tecnico inicial, design system, React/TypeScript, backend/Supabase, APIs, integracoes e revisao tecnica	Cria a fundacao antes da distribuicao das tasks e revisa banco, backend, webhooks e conectores
Anderson	Implementacao de features apos a fundacao e responsavel principal pelos testes humanos/funcionais em staging	Abre PRs e executa QA humano; codigo de risco medio/alto passa por Samuel ou Elias

Versao 1.2 - 02 de outubro de 2026

Stack de referencia: React + TypeScript + Vite, Supabase/PostgreSQL, Capacitor, Stripe/Stripe Connect e APIs externas de agendamento.



1. Como este projeto deve funcionar no dia a dia

Decisao central

Teremos UM repositorio, uma branch main protegida, branches curtas por tarefa, Pull Request obrigatorio, verificacoes automaticas, ambiente de homologacao antes da producao e revisao obrigatoria nas partes mais sensiveis. Nao usaremos uma branch permanente para cada modulo.

Tarefa / Issue

- > criar branch curta (feat/... ou fix/...)
- > desenvolver localmente com apoio do Cursor/IA
- > lint + typecheck + testes + build
- > abrir Pull Request
- > CI automatico no GitHub
- > revisao humana
- > homologacao / staging
- > merge na main protegida
- > release para producao

Regras que nao sao negociaveis

- Ninguem faz push direto na main - nem Samuel, nem Elias, nem Anderson.
- Toda mudanca de codigo entra na main por Pull Request.
- Pagamentos, autenticacao, RLS, migrations, secrets e deploy de producao exigem revisao de Samuel ou Elias.
- Mudancas de banco devem existir como migrations versionadas. Alteracao manual em producao nao e o fluxo normal.
- Local e staging usam credenciais de teste. Segredos de producao nunca entram no codigo, commit ou prompt do Cursor.
- Codigo gerado por IA segue o mesmo processo de revisao e teste de codigo escrito por uma pessoa.
- Branch e criada por tarefa. Terminou, fez merge, apaga a branch.
- Regras especificas de Acuity, Square, Wix, Zenoti e Mindbody ficam isoladas em adapters de integracao, nao espalhadas pelas telas.

Por que esse modelo

O objetivo e proteger o projeto sem criar uma burocracia de empresa gigante. A main vira nossa fonte estavel. Branches curtas diminuam conflitos. Pull Requests criam um ponto claro de revisao. O CI pega erros mecanicos. O staging evita que a producao vire laboratorio.

2. Bubble e Figma: como vamos usar o projeto antigo

Muito importante

O Detox Pass ja existe hoje no Bubble e tambem possui Figma. Os dois materiais sao referencias valiosas para entender fluxo, comportamento, campos, telas e regras ja pensadas. Mas eles NAO sao a fonte atual da identidade visual, porque o projeto mudou de marca/visual desde que foram feitos.

2.1 O que consultar no Bubble

- Fluxos que ja funcionam ou ja foram testados no produto anterior.
- Campos, cadastros, estados e regras de negocio historicas.
- Comportamento de telas e sequencia de acoes do usuario.
- Dados e estruturas que precisem ser migrados ou entendidos antes da reconstrucao.
- Casos de borda que talvez nao estejam escritos em um documento, mas aparecem no funcionamento atual.

2.2 O que consultar no Figma

- Hierarquia das telas e jornadas ja desenhadas.
- Organizacao de informacoes e componentes que ainda facam sentido.
- Fluxos de navegacao e estados de interface.
- Referencias historicas de UX que ajudem a nao reinventar o produto sem necessidade.

2.3 O que NAO copiar cegamente

- Cores, tipografia, logo, icones e estilo visual antigos.
- Layouts que conflitem com a nova identidade aprovada.



- Regras que já tenham sido alteradas em reuniões, contratos, PRD ou decisões mais recentes.
- Limitações técnicas do Bubble. A nova versão é high-code e pode resolver o mesmo problema de forma diferente.

2.4 Ordem de prioridade quando houver conflito

Prioridade	Fonte	Como tratar
1	Decisão mais recente aprovada por Samuel/Elias e escopo atual	É a regra que vale.
2	Nova identidade visual / design system aprovado	Vale para cores, tipografia, componentes e aparência.
3	Código e arquitetura high-code atuais	Vale para implementação técnica já consolidada.
4	Figma e Bubble legados	Servem como referência funcional/histórica, não como autoridade final.

Regra para humanos e Cursor

Se Bubble/Figma disserem uma coisa e uma decisão atual disser outra, vale a decisão atual. Se a dúvida não estiver resolvida, pare e pergunte. Não invente uma terceira regra.



3. Kickoff tecnico: como o projeto começa antes de dividir tarefas

Ideia central	Ninguem começa a desenvolver features em paralelo no primeiro minuto. Primeiro criamos uma fundacao comum: repositorio, documentacao, ambientes, Supabase, design system, estrutura TypeScript, backend inicial e paginas mockadas. So depois dessa base estar funcional e revisada distribuimos as tarefas.
----------------------	--

3.1 Ordem do kickoff

- Samuel cria o repositorio no GitHub, configura acessos, protecao da main, documentacoes iniciais e registra as decisoes de arquitetura.
- Samuel e Elias definem a estrutura inicial do repositorio. O padrao deste projeto continua sendo um repositorio unico; monorepo so deve ser adotado se a separacao em apps/pacotes trazer beneficio real e isso for registrado antes do desenvolvimento paralelo.
- Samuel cria ou define quais contas/projetos de infraestrutura serao usados, especialmente Supabase, ambientes, dominio, contas de terceiros e estrategia de secrets. Se houver recursos legados reaproveitaveis, isso deve ser decidido no kickoff.
- Elias assume o bootstrap tecnico inicial: cria a base React + TypeScript, configura a estrutura de pastas, prepara o backend/base Supabase, valida o design system e cria as primeiras paginas mockadas.
- Elias abre o Pull Request da fundacao. Samuel revisa as decisoes estruturais. O CI precisa passar e o ambiente de staging deve subir com a base minima funcionando.
- Somente depois do marco de fundacao aprovado as tasks de features sao distribuidas para Samuel, Anderson e Elias. Assim todos trabalham sobre a mesma referencia e evitamos tres arquiteturas diferentes nascendo ao mesmo tempo.

3.2 Responsabilidade da fundacao inicial

Responsavel	Responsabilidade no kickoff	Resultado esperado
Samuel	Repositorio, acessos, documentacao, protecao da main, decisao de estrutura, contas/ambientes e alinhamento de produto.	Um projeto governado, com regras claras e infraestrutura acessivel ao time.
Elias	Bootstrap tecnico, React/TypeScript, estrutura de codigo, design system, backend/base Supabase e paginas mockadas.	Uma primeira versao tecnica coerente que vira referencia para o restante do time.
Anderson	Acompanha o kickoff, aprende a estrutura, valida o sistema como usuario e so inicia features depois da fundacao aprovada.	Entra no desenvolvimento sem precisar adivinhar arquitetura, padroes ou componentes.

3.3 O que significa "paginas mockadas"

Paginas mockadas sao telas que ja mostram a estrutura visual e os componentes principais, mesmo que nem todos os dados reais e integracoes estejam funcionando ainda. Elas servem como um mapa visual: ajudam o time a enxergar como o produto vai se organizar antes de cada feature ficar completa.

Exemplo: a tela de agenda pode existir com dados ficticios e componentes reais do design system antes da integracao com Acuity/Square estar pronta. Depois a task de integracao conecta essa tela ao backend e aos provedores.

3.4 Marco de Fundacao (Foundation Gate)

- Repositorio criado e main protegida.
- Estrutura de pastas e padrao TypeScript definidos.
- Design system inicial validado.
- Paginas principais mockadas o suficiente para orientar o time.
- Supabase/backend inicial configurado e migrations iniciais versionadas.
- CI minimo executando lint, typecheck, testes disponiveis e build.
- Staging conseguindo subir a aplicacao base.
- Documentacao de setup suficiente para qualquer integrante clonar e executar o projeto.
- Pull Request da fundacao revisado por Samuel e mergeado na main.

**Regra de distribuicao**

Antes do Foundation Gate, o foco e construir uma base unica. Depois dele, o trabalho paralelo com Samuel e Anderson acelera de verdade porque todos partem do mesmo design system, backend e arquitetura.

4. Repositorio e organizacao do codigo

4.1 Um repositorio unico

O Detox Pass deve começar em um unico repositorio. Nele ficam a aplicacao React, os arquivos do Supabase, os projetos Capacitor Android/iOS, testes, scripts e documentacao. Isso simplifica o trabalho do time e ajuda o Cursor a enxergar o contexto inteiro do sistema.

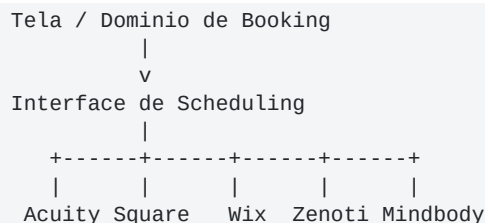
```
detox-pass/  
|-- src/  
|   |-- app/  
|   |-- components/  
|   |-- features/  
|       |-- auth/  
|       |-- providers/  
|       |-- marketplace/  
|       |-- bookings/  
|       |-- payments/  
|       |-- rewards/  
|       |-- admin/  
|   |-- integrations/  
|       |-- scheduling/  
|           |-- acuity/  
|           |-- square/  
|           |-- wix/  
|           |-- zenoti/  
|           |-- mindbody/  
|       |-- payments/stripe/  
|   |-- lib/  
|   |-- hooks/  
|   |-- types/  
|-- supabase/  
|   |-- migrations/  
|   |-- functions/  
|   |-- seed/  
|-- tests/  
|-- docs/  
|-- scripts/  
|-- android/  
|-- ios/  
|-- capacitor.config.*  
|-- .github/workflows/  
|-- .github/CODEOWNERS  
|-- README.md
```

4.2 Organizar por feature

Sempre que possivel, uma feature deve manter perto dela as telas, componentes, hooks e regras de dominio que pertencem ao mesmo assunto. Exemplo: booking fica em bookings; pagamentos ficam em payments. Componentes realmente reutilizaveis vao para areas compartilhadas. Evite arquivos gigantes de utilitarios e evite colocar regra de API externa dentro de componente visual.



4.3 Regra dos adapters de agenda



Cada provedor tenta implementar o mesmo contrato interno: consultar disponibilidade, ler booking, criar, reagendar, cancelar e tratar webhook. Se um provedor não suporta uma operação, o adapter informa explicitamente que a capacidade não existe. Não simulamos uma capacidade que a API não oferece.

5. Git: branches curtas e Pull Requests

5.1 Modelo de branch

Não teremos uma branch eterna chamada booking, pagamentos ou auth. A main deve ficar sempre pronta para deploy. O trabalho acontece em branches pequenas criadas a partir da main atualizada.

Prefixo	Quando usar	Exemplo
feat/	Nova funcionalidade	feat/booking-reschedule
fix/	Correção de bug	fix/stripe-webhook-idempotency
refactor/	Reorganização sem mudar comportamento	refactor/scheduling-adapter
chore/	CI, dependências ou configuração	chore/github-actions
docs/	Documentação	docs/cursor-rules
hotfix/	Correção urgente em produção	hotfix/login-regression

Ciclo de uma tarefa

1. Atualizar a main local e buscar a versão mais recente do GitHub.
2. Criar uma branch para uma tarefa coerente e pequena.
3. Implementar e fazer commits lógicos.
4. Executar verificações locais.
5. Enviar a branch e abrir um Pull Request.
6. Corrigir erros do CI e responder revisões.
7. Validar em staging quando a mudança altera comportamento.
8. Fazer squash merge, quando apropriado, e apagar a branch.

Evitar branches longas

Não criar branch-auth, branch-booking, branch-payments e deixar abertas por semanas. Um módulo grande deve ser construído por várias tarefas pequenas que entram continuamente na main. Isso reduz conflitos e o famoso "na minha branch funcionava".

5.2 Padrão de commits

```
feat(bookings): adiciona fluxo de reagendamento
fix(payments): evita processar webhook duplicado
refactor(scheduling): extrai interface dos provedores
chore(ci): adiciona typecheck no pull request
docs(cursor): documenta regras de pagamentos
```

Evitar commits como "update", "final", "mudanças da IA", "teste" ou "arrumei". O commit deve contar o que mudou.



6. Responsabilidades, revisao e nivel de risco

6.1 Quem faz o que

Pessoa	Foco esperado	Padrao de revisao
Samuel	Kickoff/governanca, arquitetura, produto, segurancas, pagamentos, releases e decisoes finais	Cria/organiza a base com Elias; revisa PRs criticos e resolve duvidas estruturais
Elias	Bootstrap tecnico, design system, React/TypeScript, backend, Supabase, APIs, integracoes e revisao tecnica	Entrega a primeira base funcional; revisa banco, backend, webhooks e conectores externos
Anderson	Features, telas, CRUDs, fluxos e QA humano do produto em staging	Testa manualmente as entregas; registra falhas/evidencias; partes sensiveis nao entram sem revisao

6.2 Classificacao de risco

Risco	Exemplos	Regra
Baixo	UI, textos, componentes, ajustes visuais, listagens simples	PR normal; revisao leve
Medio	Bookings, queries, regras de negocio, webhooks nao financeiros, adapters de agenda	Revisao tecnica antes do merge
Alto	Stripe, payout, refund, dispute, auth, RLS, service_role, migration, secrets, producao	Samuel ou Elias obrigatoriamente revisa

6.3 CODEOWNERS sugerido

```
/src/features/payments/    @samuel @elias
/src/integrations/payments/ @samuel @elias
/supabase/migrations/     @samuel @elias
/supabase/functions/      @samuel @elias
/.github/workflows/       @samuel @elias
```

Os handles acima sao ilustrativos. O objetivo do CODEOWNERS nao e impedir Anderson de trabalhar nessas partes. E garantir que codigo sensivel nao seja aprovado por acidente sem revisao experiente.

6.4 Tamanho de Pull Request

Prefira PRs que uma pessoa consiga entender em aproximadamente 20 a 30 minutos. Se uma tarefa esta enorme, divida verticalmente. Exemplo: primeiro leitura do perfil, depois edicao, depois validacoes. Branch gigante por muitos dias e sinal de que a tarefa esta grande demais.

7. Padrao de Pull Request

Todo PR deve explicar

- Qual problema ou tarefa ele resolve.
- O que foi alterado e quais arquivos/areas principais foram afetados.
- Como testar manualmente.
- Se existe migration, nova variavel de ambiente, webhook ou configuracao externa.
- Riscos conhecidos e pontos que precisam de atencao.
- Screenshots ou video curto quando houver mudanca visual relevante.

Checklist de revisao

- O codigo realmente atende a tarefa e aos criterios de aceite?
- Entrou refactor sem relacao com a tarefa?
- Existe secret, token, senha ou PII exposto?
- Erros e casos de borda foram considerados?
- Regra especifica de provedor esta isolada em adapter?



- RLS/permissoes estao corretas?
- Pagamento e webhook sao idempotentes e auditaveis?
- Existem testes ou passos de validacao reproduziveis?
- A feature passou pelo teste humano do Anderson em staging quando aplicavel?
- O PR registra evidencia de QA humano para fluxos de interface, booking, integracao ou pagamento?
- A mudanca pode quebrar Capacitor/mobile ou configuracao de producao?

8. CI: a esteira automatica antes do merge

Todo Pull Request para main deve rodar verificacoes automaticas. Se uma verificacao obrigatoria falhar, o merge fica bloqueado. Isso evita que um erro simples vire um problema maior.

```
npm ci
npm run lint
npm run typecheck
npm run test
npm run build
```

Scripts minimos esperados

Script	Serve para
lint	Encontrar padroes ruins e erros de codigo
typecheck	Confirmar que TypeScript esta consistente
test	Executar testes automatizados disponiveis
build	Garantir que o projeto realmente compila para entrega

A ferramenta de testes pode mudar ao longo do projeto. O importante e que o fluxo continue existindo. Se algum check ainda nao estiver disponivel, o PR deve dizer isso claramente - nao pode simplesmente pular em silencio.



9. Estrategia de testes: IA ajuda, mas Anderson valida como pessoa

Regra principal	Os agentes do Cursor e os testes automatizados procuram erros técnicos. Anderson e o responsável principal pelos testes humanos/funcionais: ele usa a funcionalidade como um usuário real, tenta caminhos normais e casos de erro e registra o resultado antes de considerarmos a entrega pronta.
------------------------	---

9.1 Camadas de teste

Camada	Quem executa	O que procura	Quando
1 - Checks automáticos	CI + agentes/ambiente local	Erros de lint, TypeScript, testes automatizados quebrados e falha de build.	Em todo PR.
2 - Testes técnicos direcionados	Quem implementou + Cursor	Regras da feature, APIs, casos de borda e regressão técnica.	Antes de pedir revisão.
3 - Teste humano funcional	Anderson	Se a pessoa consegue usar o fluxo de verdade, se a tela faz sentido e se existem comportamentos estranhos.	Em staging, antes do aceite da feature.
4 - Revisão de risco	Samuel ou Elias	Segurança, arquitetura, dados, pagamentos, RLS, migrations, APIs e riscos de produção.	Obrigatório em médio/alto risco.
5 - Smoke test de release	Anderson + responsável da release	Fluxos essenciais continuam funcionando após o conjunto de merges.	Antes e logo após uma release relevante.

9.2 Como Anderson deve fazer o teste humano

- Ler a task e os critérios de aceite antes de testar.
- Abrir a versão de staging, nunca usar produção como primeiro lugar de teste.
- Executar o caminho feliz: fazer exatamente o que um usuário comum faria quando tudo está certo.
- Executar pelo menos um caminho de erro: dado inválido, botão repetido, horário indisponível, falta de permissão ou outro caso relevante.
- Conferir desktop e mobile quando a mudança afeta interface responsiva.
- Quando houver integração externa, conferir não apenas a tela, mas também se o efeito aconteceu no sistema externo ou nos logs/eventos esperados.
- Registrar resultado: APROVADO ou REPROVADO. Se reprovado, informar passos para reproduzir, resultado esperado, resultado obtido e anexar screenshot/video quando ajudar.
- Depois da correção, repetir o mesmo teste. Não assumir que "deve ter resolvido".

9.3 Evidência de QA humano

Para features de interface, booking, pagamentos, agenda externa ou fluxo crítico, o PR deve conter uma seção curta de QA humano com quem testou, ambiente, data, cenários executados e resultado. Anderson e o responsável padrão por esse registro, salvo combinação diferente na task.

Campo	Exemplo
Responsável	Anderson
Ambiente	staging
Cenário 1	Criar booking com horário válido - APROVADO
Cenário 2	Tentar horário já ocupado - APROVADO
Cenário 3	Cancelar booking - REPROVADO: webhook não atualizou status
Evidência	link para screenshot/video ou comentário no PR



9.4 O Cursor nao substitui esse teste

O Cursor pode escrever testes, executar comandos, analisar logs e sugerir casos de borda. Isso e excelente e deve ser usado. Mas ele nao substitui a validacao humana de experiencia e comportamento. Se o agente disser "todos os testes passaram", isso significa apenas que os checks executados passaram - nao que uma pessoa testou o produto ponta a ponta.

10. Ambientes e deploy

10.1 Fluxo de ambientes

```
LOCAL / DESENVOLVIMENTO
  |
  v
STAGING / HOMOLOGACAO
  |
  v
PRODUCAO
```

10.2 Regras de deploy

- Deploy de producao sai da main, nunca de uma branch aleatoria.
- Mudancas de alto risco precisam passar por staging antes da producao.
- Nunca testar payout/refund real pela primeira vez em producao.
- Criar tags/releases importantes para termos um ponto conhecido de rollback.
- Se producao quebrar, primeiro estabilizar/voltar. Investigar com calma depois.

10.3 Principio de rollback

Aplicacao: voltar para uma release/commit conhecido. Banco: usar migration de correcao ou plano de restauracao; nao improvisar SQL destrutivo em producao. Migrations de dados precisam considerar reversibilidade, backup e impacto.

11. Supabase e banco de dados

11.1 Migrations

```
supabase/migrations/
  20261002_create_bookings.sql
  20261003_add_provider_status.sql
  20261004_add_payment_ledger.sql
```

- Nao alterar tabela de producao manualmente como rotina.
- Nao apagar coluna/tabela nem reescrever dados de producao sem aprovacao e plano de backup/rollback.
- Quando possivel, preferir migrations aditivas primeiro.
- Seed e para dados de teste/desenvolvimento, nao para gambiarra em producao.
- Mudanca em RLS e risco alto e exige revisao.
- service_role e somente server-side. Nunca expor no React ou bundle mobile.
- Nova variavel de ambiente precisa ser documentada no PR sem revelar seu valor secreto.

11.2 Acesso a dados

As telas nao devem espalhar queries cruas por todo lado. Prefira services, repositories ou hooks bem definidos. Isso facilita revisar permissoes, trocar integracoes e entender o impacto de migrations.

12. Pagamentos, seguranca e APIs externas

12.1 Stripe / Stripe Connect

- Usar Stripe em modo de teste no local e staging.
- Webhooks devem validar assinatura e ser idempotentes.
- O status do frontend nao e a fonte de verdade do pagamento; confirmar no backend/eventos Stripe.
- Payout, refund e dispute sao risco alto e exigem revisao.
- Nao armazenar dados completos de cartao no Detox Pass.



- Não hardcodar secret keys ou IDs sensíveis.
- Ações financeiras de produção precisam deixar trilha auditável com IDs internos/externos e histórico de status.

12.2 APIs de agenda

- Não assumir que todos os provedores fazem as mesmas coisas.
- Cada adapter deve expor capacidades reais e operações não suportadas.
- Revalidar disponibilidade antes de confirmar booking.
- Usar webhooks quando existirem e reconciliação quando fizer sentido.
- Não implementar scraping que viole termos de terceiros.
- Documentação/sandbox funcionando não significa homologação em produção.

12.3 Secrets

Nunca coloque segredo no prompt do Cursor

API keys, service_role, Stripe secret key, certificados, senhas, tokens e dados reais de usuários não devem ser colados em prompts nem commitados. Use placeholders e variáveis de ambiente. Se um segredo vazar, trate como comprometido e rotacione.

13. Regras para Cursor e agentes de IA

Esta seção deve ser tratada como instrução direta aos agentes de IA quando este PDF estiver anexado no Cursor.

13.1 Antes de mudar código

- Leia a tarefa, os critérios de aceite e este playbook antes de editar.
- Inspecione a implementação atual e siga os padrões existentes antes de criar uma arquitetura paralela.
- Identifique o menor conjunto de arquivos necessário para a tarefa.
- Declare suposições. Se uma decisão de arquitetura, segurança ou pagamento estiver incerta, pare e pergunte.
- Não faça limpeza/refactor geral que não foi solicitado.
- Quando precisar entender comportamento legado, consulte Bubble e Figma como referência funcional; não copie a identidade visual antiga.

13.2 Enquanto codifica

- Trabalhe apenas na branch da tarefa; nunca sugira push direto na main.
- Preserve tipagem TypeScript. Não silencie erro com any, ts-ignore ou lint desativado sem justificativa.
- Mantenha regra de negócio fora de componentes puramente visuais quando possível.
- Use a abstração de Scheduling Adapter para agendas externas.
- Operações privilegiadas e secrets ficam em fronteiras server-side.
- Não exponha service_role nem Stripe secret key no frontend/mobile.
- Não altere dados de produção nem rode comando destrutivo sem aprovação humana explícita da operação exata.
- Ao adicionar dependência, explique por que ela é necessária e evite bibliotecas duplicadas para o mesmo problema.
- Não invente capacidade de API. Se documentação/credencial não comprova suporte, marque como pendente/não suportado.

13.3 Antes de declarar uma tarefa pronta

- Rodar lint, typecheck, testes e build - ou informar exatamente o que não conseguiu rodar e por que.
- Resumir arquivos alterados e comportamento novo.
- Avisar sobre migration, nova env, webhook ou configuração externa necessária.
- Fornecer passos de teste manual para fluxos de UI/integração.
- Confirmar que nenhum secret, credencial de debug ou PII real foi incluído.
- Não fazer merge sozinho; deixar aprovação e merge para o fluxo do time.
- Não declarar "testado pelo usuário" sem teste humano real. Quando aplicável, sinalizar no resumo: "Aguardando QA humano do Anderson em staging".

13.4 Regra especial para áreas de alto risco

IA pode implementar; IA não pode se autoaprovar

Cursor pode escrever código de pagamentos, auth, RLS, migrations e webhooks. Mas essas mudanças precisam de revisão humana antes do merge. Velocidade de geração não substitui responsabilidade técnica.



13.5 Cabecalho sugerido para prompts no Cursor

Contexto obrigatorio:

- Siga o Detox Pass Playbook de Desenvolvimento anexado.
- Respeite a fundacao tecnica criada no kickoff; nao crie um segundo design system, backend paralelo ou arquitetura concorrente sem aprovacao.
- Respeite a identidade visual atual; Bubble/Figma sao referencia historica de fluxo.
- Nao faca push direto na main.
- Nao invente capacidades de APIs externas.
- Nao exponha secrets.
- Minimize o escopo da mudanca.
- Antes de concluir, rode/checke lint, typecheck, testes e build.
- Quando a mudanca exigir validacao funcional, marque explicitamente: "Aguardando QA humano do Anderson em staging".
- Para pagamentos, auth, RLS, migrations ou producao, sinalize revisao humana obrigatoria.



14. Definition of Done - quando uma tarefa esta realmente pronta

Uma tarefa nao termina porque a tela "parece pronta". Ela termina quando os itens aplicaveis abaixo foram cumpridos.

- Criterios de aceite atendidos.
- Codigo revisavel, sem mudancas aleatorias fora da tarefa.
- TypeScript sem erros conhecidos.
- Lint aprovado.
- Testes relevantes executados.
- Build concluido.
- Migration criada quando houve mudanca de banco.
- RLS/permissoes revisadas quando aplicavel.
- Env vars/configuracoes documentadas.
- Fluxo validado em staging quando aplicavel.
- Teste humano funcional executado por Anderson, quando a tarefa altera comportamento visivel, booking, integracao ou fluxo de negocio.
- Falhas encontradas no QA humano corrigidas e retestadas antes do aceite.
- Nenhum secret ou dado real sensivel no repositorio.
- PR aberto com explicacao e passos de teste.
- Revisao obrigatoria concluida nas areas de medio/alto risco.

15. Formato padrao de tarefa - humano e IA recebem igual

TITULO: [acao + objeto]
OBJETIVO: [o que queremos atingir]
CONTEXTO: [por que existe]
ESCOPO:
- item 1
- item 2
FORA DO ESCOPO:
- item 1
CRITERIOS DE ACEITE:
- criterio 1
- criterio 2
REFERENCIAS:
- Bubble: [tela/fluxo, se aplicavel]
- Figma: [frame/fluxo, se aplicavel]
- identidade visual atual: [referencia]
RISCO: baixo | medio | alto
TESTES AUTOMATIZADOS ESPERADOS: [passos/checks]
QA HUMANO (Anderson): [cenarios que precisam ser validados em staging]
OBSERVACOES: [migrations/env/API]

Uma boa tarefa reduz retrabalho mais do que um prompt enorme. O Cursor deve receber objetivo, limites e criterios claros.

16. Procedimento de incidente / hotfix

1. Parar novos deploys se o problema puder piorar dados, pagamentos ou seguranca.
2. Criar uma branch hotfix a partir do estado atual de producao/main.
3. Reproduzir e identificar a menor correcao segura.
4. Em pagamento, seguranca ou dados, Samuel ou Elias revisa antes do deploy.
5. Rodar CI e testes direcionados.
6. Fazer deploy do hotfix, verificar saude de producao e registrar o ocorrido.
7. Criar tarefa posterior para causa raiz se o patch emergencial for propositalmente minimo.

17. O que NAO fazer

- Nao fazer push direto na main.
- Nao manter branch por semanas so porque ela representa um modulo.
- Nao editar banco de producao sem migration e sem plano.
- Nao deixar o Cursor refatorar partes nao relacionadas "ja que esta aqui".



- Não commitar .env, keys, certificados ou secrets.
- Não usar service_role no client para contornar RLS.
- Não testar payout/refund real antes do fluxo de teste/staging.
- Não aprovar PR apenas porque a IA disse que está correto.
- Não duplicar lógica de provedor em telas diferentes.
- Não prometer operação de API externa antes de validar autenticado.
- Não copiar identidade visual antiga do Bubble/Figma para a nova versão.

18. Comandos de referência rápida

Atualizar main e criar branch

```
git checkout main
git pull origin main
git checkout -b feat/nome-da-tarefa
```

Antes de abrir PR

```
npm run lint
npm run typecheck
npm run test
npm run build
```

Enviar branch

```
git add .
git commit -m "feat(area): descricao objetiva"
git push -u origin feat/nome-da-tarefa
```

Importante

Os comandos acima são referência. Nunca rode comando destrutivo, de produção ou de banco apenas porque o Cursor sugeriu. Entenda o que o comando faz ou peça revisão.

19. Checklist de início do projeto

- Samuel cria o repositório, define acessos e protege a main.
- Samuel e Elias registram a decisão de estrutura: repo único por padrão; monorepo apenas se houver justificativa real.
- Criar ou definir projetos/contas de Supabase e ambientes; registrar como secrets serão gerenciados.
- Configurar CODEOWNERS, revisores e CI mínimo: lint, typecheck, test e build.
- Disponibilizar Bubble e Figma como referência histórica e a nova identidade visual como autoridade atual.
- Elias faz o bootstrap: React + TypeScript, estrutura de pastas, design system, backend/Supabase base e páginas mockadas.
- Abrir PR da fundação, revisar, rodar CI e publicar a base em staging.
- Somente após o Foundation Gate distribuir features entre Samuel, Anderson e Elias.
- Definir primeiro conector de agenda para POC e configurar Stripe em test mode/webhook de staging.
- Anderson prepara o checklist de QA humano e passa a testar as features em staging.
- Anexar este playbook ao Cursor e usar o cabeçalho padrão nos prompts.

Resumo final

Um repo. Main protegida. Branch curta por tarefa. PR obrigatório. CI automático. Staging antes de produção. Banco por migration. Áreas críticas com revisão. Bubble e Figma como memória funcional, não como identidade visual atual. IA acelera o trabalho, mas não substitui critério, revisão e responsabilidade.